

Ejemplo RecyclerView

Añadimos el recyclerView en nuestro layout:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerViewPelículas"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginHorizontal="8dp"
    android:layout_marginVertical="16dp" />
```

*** La altura del contenedor será android:layout_height="wrap_content"

Cada elemento de la lista mostrará dos datos, el nombre de la película y el director. Para ello crearemos un nuevo layout fila_película.xml en la carpeta /app/res/layout. Necesitamos un LinearLayout

```
<TextView
    android:id="@+id/filaTitulo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="8dp"
    android:layout_marginTop="8dp"
    android:text="Titulo"
    android:textSize="30sp"
    android:textStyle="bold"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

<TextView
    android:id="@+id/filaDirector"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="8dp"
    android:layout_marginTop="8dp"
    android:textSize="15sp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/filaTitulo"
    android:text="Director" />
```

*** La altura del contenedor será android:layout_height="wrap_content"

Para manejar mejor los datos, crearemos una clase que encapsulará los dos datos de cada elemento. Le llamamos Pelicula:

```
class Pelicula (titulo: String, director: String){
    var titulo: String
    var director:String
    init {
        this.titulo = titulo
        this.director=director
    }
}
```

La clase RecyclerView.Adapter nos obligará de cierta forma a hacer uso del patrón view holder. Por tanto, lo primero que definiremos será el ViewHolder. Lo definiremos como clase interna a nuestro adaptador, extendiendo de la clase RecyclerView.ViewHolder, y tan sólo tendremos que incluir como atributos las referencias a los controles del layout de un elemento de la lista (en nuestro caso los dos TextView) e inicializarlas en el constructor utilizando como siempre el método findViewById() sobre la vista recibida como parámetro.

```
class ViewHolderPelicula(itemView: View) :
RecyclerView.ViewHolder(itemView) {
    val textViewTitulo: TextView
    val textViewDirector: TextView

    init {
        textViewTitulo = itemView.findViewById(R.id.filaTitulo)
        textViewDirector = itemView.findViewById(R.id.filaDirector)
    }
}
```

El siguiente paso a realizar es escribir nuestro adaptador. Este adaptador deberá de extender de la clase RecyclerView.Adapter y tendremos que sobrescribir los siguientes tres métodos:

- onCreateViewHolder(). Encargado de crear los nuevos objetos ViewHolder necesarios para los elementos de la colección.
- onBindViewHolder(). Encargado de actualizar los datos de un ViewHolder ya existente.
- getItemCount(). Indica el número de elementos de la colección de datos.

Ahora crearemos una nueva Clase de tipo Kotlin y se llamará AdaptadorPeliculas.kt

```
class AdaptadorPeliculas(datosPelis: MutableList<Pelicula>) :
RecyclerView.Adapter<ViewHolderPelicula>() {
```

```

private lateinit var peliculas: MutableList<Película>

init {
    this.peliculas=datosPelis
}

override fun onCreateViewHolder(parent: ViewGroup, viewType:
Int): ViewHolderPelícula {
    // inflamos la vista (fila_pelicula.xml)
    val vista: View =
LayoutInflater.from(parent.context).inflate(R.layout.fila_pelicula,
parent, false)
    // Crear el viewholder a partir de esta vista.
    val viewHolder = ViewHolderPelícula(vista)
    return viewHolder
}

override fun getItemCount(): Int {

    return this.peliculas.size
}

override fun onBindViewHolder(holder: ViewHolderPelícula,
position: Int) {
    // Dibujar la fila de la peli con los datos de la peli
    actualmente solicitada según la variable position
    val peli: Película = this.peliculas[position]
    holder.textViewTitulo.text = peli.titulo
    holder.textViewDirector.text=peli.director
    Log.d("recycler", position.toString());
}
}

```

Con esto tendríamos finalizado el adaptador, por lo que ya podríamos asignarlo al RecyclerView en nuestra actividad principal. Tan sólo tendremos que crear nuestro adaptador personalizado AdaptadorPeliculas pasándole como parámetro la lista de datos y asignarlo al control RecyclerView mediante su propiedad adapter.

```

recyclerViewpeliculas = findViewById(R.id.recyclerViewPeliculas)
recyclerViewpeliculas.layoutManager =
LinearLayoutManager(this)
// La línea que divide los elementos
recyclerViewpeliculas.addItemDecoration(
    DividerItemDecoration(

```

```

        this@MainActivity,
        LinearLayoutManager.VERTICAL
    )
)
// El adaptador que se encarga de toda la lógica
recyclerViewpeliculas.setAdapter(adaptadorRecyclerView)
}

```

Si deseais probar con datos reales modificad la creación del adaptador por:

```

val peliculas= ArrayList<Pelicula>()
peliculas.add(Pelicula("Star Wars", "George Lucas"))
peliculas.add(Pelicula("Thor", "Taika Waititi"))
peliculas.add(Pelicula("Iron Man", "Jodatos_pelisn Favreau"))
peliculas.add(Pelicula("Los vengadores", "James Gunn"))
peliculas.add(Pelicula("Los juegos del hambre", "Gary Ross"))
peliculas.add(Pelicula("Oppenheimer", "Christopher Nolan"))
peliculas.add(Pelicula("Saw", "James Wan"))
peliculas.add(Pelicula("Dune", "Denis Villeneuve"))
peliculas.add(Pelicula("El padrino", "Francis Ford Coppola"))
peliculas.add(Pelicula("El lobo de Wall Street", "Martin Scorsese"))
peliculas.add(Pelicula("El exorcista", "Francis Ford Coppola"))
peliculas.add(Pelicula("The Batman", "Matt Reeves"))

val adaptadorRecyclerView = AdaptadorPeliculas(peliculas)

```

Mejorando nuestro código RecyclerView

Para seguir manteniendo organizado y separado nuestro código, crearemos una función llamada bind() en nuestro ViewHolder personalizado, que se encargara de unir los datos con la vista. Para ello recibirá un objeto de tipo Pelicula

```

fun bind(peli: Pelicula) {
    textViewTitulo.text=pele.titulo
}

```

```
textViewDirector.text=pele.director  
}
```

Y que será llamado desde nuestro adaptador

```
val peli: Pelicula = this.peliculas[position]  
  
//holder.textViewTitulo.text = String.valueOf(peli.titulo)  
  
//holder.textViewDirector.text = String.valueOf(peli.director)  
  
holder.bind(peli)
```

Añadir cardview

```
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"  
xmlns:tools="http://schemas.android.com/tools"  
xmlns:card_view="http://schemas.android.com/apk/res-auto"  
android:orientation="vertical"  
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
android:layout_marginVertical="8dp"  
android:layout_marginHorizontal="8dp"  
card_view:cardBackgroundColor="#aaa"  
android:elevation="1dp"  
card_view:cardCornerRadius="10dp"  
android:layout_margin="8dp">  
  <LinearLayout  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:orientation="vertical">  
    <TextView  
      android:id="@+id/filaTitulocd"  
      android:layout_width="wrap_content"  
      android:layout_height="wrap_content"  
      android:layout_marginStart="8dp"  
      android:layout_marginTop="8dp"  
      android:text="Titulo"  
      android:textSize="30sp"  
      android:textStyle="bold" />  
    <TextView  
      android:id="@+id/filaDirectorcd"  
      android:layout_width="wrap_content"  
      android:layout_height="wrap_content"  
      android:layout_marginStart="8dp"  
      android:layout_marginTop="8dp"  
      android:text="Director"  
      android:textSize="15sp" />  
    </LinearLayout>  
  </androidx.cardview.widget.CardView>
```